NES-VMC 算法的核心思想正是将原系统前K个激发态的求解问题，等价地转化为一个“扩展系统”的基态求解问题。 我希望你完全基于 Netket 框架来实现 NES-VMC 算法。 拓展系统的 HilbertSpace 已经实现 ， 只需要在采样器中使用即可。 我希望你使用 MCState 来采样拓展系统的状态。并使用拓展系统的哈密顿量来计算基态能量。 如何获得原始系统的基态、激发态能量呢？ 你可以得到拓展系统的基态能量之后, 计算出拓展系统的局域能量矩阵，对角化之后可以得到前 K 个能态能量. 以下是对 H2 分子系统为例子的实现 基于 Netket 与 Flax.nnx框架 首先`hi_K = nk.hilbert.TensorHilbert(hi**K)`是指的拓展 Hilbert 空间。 原系统的 Hilbert 空间是 `hi`，拓展后的 Hilbert 空间是 `hi_K`。 `hi.all_states()` 每个组态是 (4,) 的向量，对应着 [α0 α1 β0 β1]-> x_1。 `hi_K.all_states()` 每个组态是 (4*K,) 的向量，对应着 $x^1,x^2,...x^k$。 例如 K=2 的时候 `hi_K.all_states()[0]` >>`Array([0, 1, 0, 1, 0, 1, 0, 1], dtype=int8)` 对应着 $x^1=[0,1,0,1],x^2=[0,1,0,1]$ 原系统共有 4 个合法组态`[0, 1, 0, 1]`,`[0, 1, 1, 0]`,`[1, 0, 0, 1]`,`[1, 0, 1, 0]`, 组态顺序是：[α0 α1 β0 β1] 然而为了能够顺利地进行 NES-VMC 采样，我们需要一次性采样K个样本。以 K=2 为例子, 每次我们需要采样2个组态. 在我的例子中 我使用了`sampler = nk.sampler.MetropolisSampler(hi_ensemble, rule=tensor_rule, n_chains=16, sweep_size=32)`

```
import jax
import jax.numpy as jnp
import netket as nk
import netket.experimental as nkx
import numpy as np
from pyscf import gto, scf, fci
from flax import nnx

#==============================================================================
# 1. 全局参数 & H₂ 分子定义
#==============================================================================
K = 2  # NES-VMC 要计算的低激发态数量（基态+1个激发态）
bond_length = 1.4
geometry = [('H', (0., 0., 0.)), ('H', (bond_length, 0., 0.))]
mol = gto.M(atom=geometry, basis='STO-3G', verbose=0)
mf = scf.RHF(mol).run(verbose=0)

# FCI 精确基准（4个态）
cisolver = fci.FCI(mf)
cisolver.nroots = 4
E_fcis, fcivec = cisolver.kernel()
print("="*60)
print("H₂ FCI 基准能量")
print("="*60)
for i, e in enumerate(E_fcis):
    exc = (e - E_fcis[0]) * 27.2114
    print(f"E{i} = {e:.8f} Ha  |  激发能: {exc:.4f} eV")

# 转为 NetKet 哈密顿量
ha = nkx.operator.from_pyscf_molecule(mol)

# 1. 定义原始 Hilbert 空间（同之前）
hi = nk.hilbert.SpinOrbitalFermions(
    n_orbitals=2,
    s=1/2,
    n_fermions_per_spin=(1,1),
)
# 2. 定义 NES-VMC 参数
K = 2  # 想要同时采样的组态数
# 3. 修正点：使用指数运算符 ** 来构建扩展 Hilbert 空间
```

```
hi_ensemble = hi ** K
# 4. 后续的采样器设置保持不变
edges = [(0, 1), (2, 3)] #edges 是对 hi 的 edges
g = nk.graph.Graph(edges=edges)
# hi_ensemble.all_states()[0].shape # (12,)

hi_K = nk.hilbert.TensorHilbert(hi**K)
hi_K.all_states().shape #(4**K,4*K)

single_rule = nk.sampler.rules.FermionHopRule(hilbert=hi, graph=g)
rules_tuple = tuple(single_rule for _ in range(K))
tensor_rule = nk.sampler.rules.TensorRule(hilbert=hi_K, rules=[single_rule]*K)
sampler = nk.sampler.MetropolisSampler(hi_ensemble, rule=tensor_rule, n_chains=16, sweep_size=32
```

接下来要设计原文中对应的Single-State Ansatz $\psi(x)$ 需要注意的是 Netket 里的采样器接受的只能是 $log(\psi(x))$，不能是 $\psi(x)$。 NESTotalAnsatz 对应着原文中的 $\Psi(x^1,x^2,...,x^k) = \det(M)$ $$\Psi(\mathbf{x}^1, \dots, \mathbf{x}^K) \triangleq \det \begin{pmatrix} \psi_1(\mathbf{x}^1) & \dots & \psi_K(\mathbf{x}^1) \ \vdots & & \vdots \ \psi_1(\mathbf{x}^K) & \dots & \psi_K(\mathbf{x}^K) \end{pmatrix} $$

```
class SingleStateAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals: int, hidden_dim=16, *, rngs: nnx.Rngs):
        super().__init__()
        self.linear1 = nnx.Linear(n_spin_orbitals, hidden_dim, rngs=rngs, param_dtype=complex)
        self.linear2 = nnx.Linear(hidden_dim, hidden_dim, rngs=rngs, param_dtype=complex)
        self.output = nnx.Linear(hidden_dim, 1, rngs=rngs, param_dtype=complex)

    def __call__(self, x):
        h = nnx.tanh(self.linear1(x.astype(complex)))
        h = nnx.tanh(self.linear2(h))
        out = self.output(h)
        #log_out = jnp.log(out)
        return jnp.squeeze(out)

class NESTotalAnsatz(nnx.Module):
    def __init__(self, n_spin_orbitals, n_states=K, hidden_dim=16):
        super().__init__()
        self.n_states = n_states
        self.n_spin_orbitals = n_spin_orbitals

        # 1. 生成一串不同的 JAX 随机 key
        key = jax.random.key(42)          # 主种子
        keys = jax.random.split(key, n_states)  # 拆成 n_states 个不同 key

        # 2. 给每个 ansatz 分配不同的 key → 不同的 nnx.Rngs
        # ✅ 这是 NNX 官方支持的写法
        self.single_ansatz_list = [
            SingleStateAnsatz(
                n_spin_orbitals,
                hidden_dim,
                rngs=nnx.Rngs(keys[i])  # 每个用不同 key
            )
            for i in range(n_states)
        ]
    def __call__(self, x_batch):
        '''
        x_batch: (K, n_spin_orbitals)
        '''
        K_state = self.n_states
        M = []
        for i in range(K_state):
            row = [self.single_ansatz_list[j](x_batch[i]) for j in range(K_state)]
```

```
            M.append(jnp.array(row))
        M = jnp.stack(M)
        psi_total = jnp.linalg.det(M)
        return psi_total, M
```

由于原文里的局域能量计算$E_L(\mathbf{x}) \triangleq \Psi^{-1}(\mathbf{x}) \hat{H} \Psi(\mathbf{x})$, 其中$\hat{H}$是(4,4)维的哈密顿量矩阵. $$\hat{H}\Psi(\mathbf{x}) \triangleq \begin{pmatrix} \hat{O}\psi_1(\mathbf{x}^1) & \dots & \hat{O}\psi_K(\mathbf{x}^1) \ \vdots & & \vdots \ \hat{O}\psi_1(\mathbf{x}^K) & \dots & \hat{O}\psi_K(\mathbf{x}^K) \end{pmatrix}$$